# AI Placement Mentor: Resume Analysis, Constraint-Based Company Matching and Mock Interview Evaluation with Graceful Degradation

Mrs. B. Deepa
Assistant Professor (SG)
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
deepa.b@rajalakshmi.edu.in

DURAIMADHAN MM
Roll No. 240701131
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
duraimadhan.mm.2024.cse@rajalakshmi.edu.in

GOKUL S
Roll No. 240701148
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
gokul.s.2024.cse@rajalakshmi.edu.in

*A. Abstract*

Preparing for campus recruitment is difficult for many students. They rarely know how well their resume matches real job requirements, which roles suit their skills, or how they will perform in an interview, and personal feedback from mentors is limited by time. This study presents AI Placement Mentor, a full-stack placement preparation platform that guides a student from resume upload to mock interview practice. A FastAPI back end parses an uploaded PDF resume, extracts contact details and skills, and reports strengths, weaknesses, missing competencies and recommended roles. The extracted skills are matched against a database of companies and job roles using skill similarity and a constraint-propagation algorithm, which removes company roles whose core requirements are not met and ranks the remaining ones. A mock interview engine supports aptitude, coding, technical, behavioural and communication rounds. Answers are evaluated by the OpenAI Chat Completions API when a key is configured and by a local keyword-and-quality evaluator otherwise, so the platform keeps working without external access. A Tavus integration starts a conversational AI interviewer with a custom persona. The system is implemented with Next.js, React and TypeScript on the front end and FastAPI, SQLAlchemy and SQLite on the back end.

**Keywords:** *Resume analysis, Constraint propagation, Company matching, Mock interview, FastAPI, Next.js, OpenAI, Tavus*

## *I. INTRODUCTION*

Campus recruitment is a decisive stage in a student's career. Before an offer is made, a student usually passes through several steps: resume screening, an aptitude test, a coding round, a technical interview and a behavioural or communication round. Each step tests a different ability, and each company asks for a different combination of skills.

Most students prepare for these steps with little feedback. A resume is often written once and never checked against real job requirements, so a student may apply to roles for which a mandatory skill is missing, or overlook roles that would suit the profile well. Mock interviews depend on the time of seniors and faculty, which is limited near the placement season, and generic online advice is not tied to the student's own resume.

Artificial intelligence can close part of this gap. Language models can read an answer and comment on it, and classical AI methods can reason over skill requirements in a transparent way. Constraint satisfaction, constraint propagation and knowledge-based reasoning [10] are applied here, together with a modern language model, to a practical placement assistant.

Given a resume in PDF format, a set of companies in which each company role has a list of required skills, and a set of interview round types, the system must extract the skills and contact details from the resume, assess strengths and weaknesses, recommend suitable roles and produce a ranked list of company matches. It must also let the student practise interview rounds and receive a score and written feedback on each answer. The system must remain usable when the external language-model service is not available, and it must let each student create an account and sign in.

The objectives of the work are as follows:

- To build a web platform in which a student can register, sign in and upload a resume in PDF format.
- To extract skills and contact details from the resume and report strengths, weaknesses and recommended roles.
- To match the resume against stored companies and roles using skill similarity and a constraint-propagation algorithm, and to rank the results.
- To support mock interview rounds in aptitude, coding, technical, behavioural and communication areas, with a score and feedback for each answer.
- To evaluate answers with the OpenAI API when it is available and with a local evaluator when it is not, so that the platform works in both cases.
- To start a conversational AI interview with a configurable interviewer persona through the Tavus API, and to present all features in a responsive dashboard.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

The system covers one deployment of the platform for students who prepare for campus recruitment. Resumes are accepted in PDF format, company data is stored in a seeded SQLite database, and the interface is designed for the student role. The main contributions of this work are:

- An end-to-end pipeline that takes a PDF resume and returns skills, strengths, weaknesses, recommended roles and ranked company matches.
- A company-matching step that applies constraint propagation to remove unsuitable company roles before the rest are ranked by skill similarity.
- A dual-mode answer evaluator that uses the OpenAI Chat Completions API when available and falls back to a local keyword-and-quality evaluator.
- An integration with the Tavus API that starts a conversational interview with a custom AI interviewer persona.
- A responsive dashboard with Home, Resume Analysis and Mock Interviews sections.

## *II. LITERATURE SURVEY*

This survey covers work on resume analysis and job matching, constraint satisfaction, language-model based evaluation and robust service design, which are the four areas this project draws on.

### *A. Resume Analysis and Job Matching*

Matching a person's profile to a set of items is a standard problem in information retrieval. Jones introduced inverse document frequency, which gives more weight to rare terms [5], and Salton and Buckley compared term-weighting schemes for automatic text retrieval [11]. These ideas underlie most keyword-based and similarity-based matching, including the matching of a resume to a job description.

Ricci, Rokach and Shapira describe content-based recommendation, in which the attributes of an item are compared with a profile of the user [9]. Matching the skills in a resume with the skills required by a company role is a content-based recommendation of this kind. A weakness of similarity alone is that it can rank a role highly even when one mandatory requirement is missing, because a high overlap on other skills hides the gap. This project therefore adds a constraint step before ranking.

### *B. Constraint Satisfaction and Propagation*

A constraint satisfaction problem (CSP) is defined by a set of variables, a domain of values for each variable and a set of constraints [10]. Besides search, constraint propagation uses the constraints to remove values from the domains that cannot be part of any solution, so that later steps work on a smaller problem [10]. The theoretical basis comes from Mackworth, who introduced consistency algorithms for networks of relations [7]; Tsang and Dechter give book-length treatments of constraint satisfaction and processing [12], [3]. Here the same idea is used in a simple form: company roles whose core skill requirements are not satisfied by the resume are removed from the candidate set before any ranking is done.

### *C. Language Models for Evaluating Answers*

Brown et al. showed that large language models can perform new tasks from a few examples in the prompt, which makes them usable for judging a written answer without task-specific training [2]. OpenAI reported the capabilities of GPT-4 on professional and academic benchmarks [8]. Zheng et al. studied strong language models as judges of other model outputs and reported that GPT-4 agrees with human judgements on more than 80% of comparisons, about as often as humans agree with each other, while also describing biases such as a preference for longer answers [13]. These findings support using a language model to give feedback on interview answers, and show why a simple and transparent local check is useful as a fallback and as a sanity filter.

### *D. Service Design and Graceful Degradation*

The platform is built as a REST service, an architectural style in which resources are accessed through a uniform interface [4]. The JSON Web Token format is widely used to carry authentication claims between a client and a server [6], and SQLAlchemy is the object-relational mapper used here to store data in a relational database [1]. A further design principle is graceful degradation: a feature that depends on an external service should still return a useful result when that service is unavailable. The mock interview evaluator follows this principle by falling back to local scoring when the OpenAI service cannot be used.

### *E. Research Gap*

**TABLE I. COMPARISON OF APPROACHES**

| Approach | Strength | Limitation | Use here |
|---|---|---|---|
| Advice from seniors or placement cell | Personal and experienced | Limited by time; not available on demand | Complemented by automated feedback |
| Keyword-only resume screening | Simple and fast | Ignores mandatory requirements; gives no reason | Replaced by propagation plus similarity |
| Constraint propagation with similarity ranking | Respects core requirements, ranks the rest, easy to explain | Needs well-structured skill lists for each role | Core of company matching |
| Language model used alone | Natural feedback on free text | Needs a key and network access; may be inconsistent | Used when available, with a local fallback |

Many placement tools address a single step, such as resume checking or question practice. Few combine resume analysis, requirement-aware company matching, multi-round interview practice and a conversational interviewer in one workflow that keeps working without an external AI service. This project addresses that gap by integrating all of these in one platform.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

## *III. METHODOLOGY FOR RESUME ANALYSIS AND COMPANY MATCHING*

### *A. Overview*

The proposed system is a three-tier web application. A Next.js front end talks to a FastAPI back end over a REST interface that exchanges JSON. The back end calls four intelligence components, namely the resume agent, the company-matching algorithm, the interview evaluator and the Tavus client, and stores its data in a SQLite database through SQLAlchemy. Fig. 1 shows the architecture.

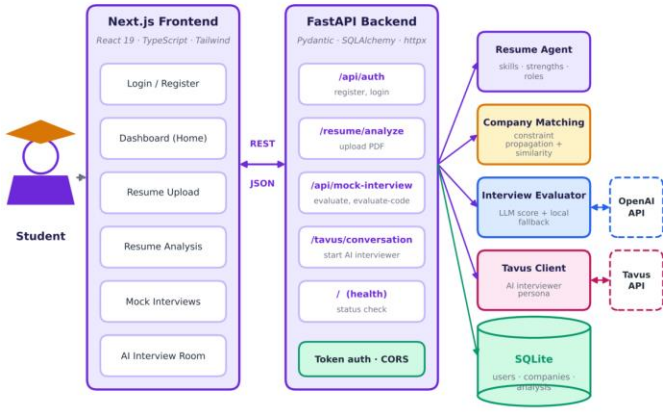



**Fig. 1. System architecture of AI Placement Mentor**

The architecture has four layers. The presentation layer uses Next.js 16 with React 19, TypeScript and Tailwind CSS, with Framer Motion for animation and Lucide for icons. The application layer is FastAPI with route groups for authentication, resume analysis, mock interview evaluation and the Tavus integration, plus a root status route, and Pydantic models validate every request. The intelligence layer holds the resume agent, the constraint-propagation matcher, the evaluator and the Tavus client. The data layer is SQLAlchemy with SQLite, which stores users, companies and resume analysis records in the file backend/data/placement.db.

**TABLE II. TECHNOLOGY STACK**

| Layer | Technology | Purpose |
|---|---|---|
| Front end | Next.js 16, React 19, TypeScript, Tailwind CSS, Framer Motion, Lucide icons | User interface, pages and dashboard |
| Back end | Python, FastAPI, Pydantic, httpx, python-dotenv | REST API, validation, outgoing HTTP calls, configuration |
| Security | Token-based authentication | Sign-in and access to the API |
| Database | SQLite with SQLAlchemy | Persistent storage of users, companies, analyses |
| AI engine | Resume agent and constraint-propagation matcher | Skill analysis and company matching |
| Language model | OpenAI Chat Completions API, with local fallback | Evaluation of interview answers |
| Video interview | Tavus API | Conversational AI interviewer |

### *B. Database Design*

Fig. 2 shows the logical tables and the main fields of each. The users table stores the account details entered at registration, including personal academic information. The companies table stores each company with its roles and the skills required for each role; it is filled by the seeding script seed_companies.py and is the only source of company data for matching. The resume_analysis table stores the result of each analysis, including the extracted contact details, skills, strengths, weaknesses, recommended roles and company matches.

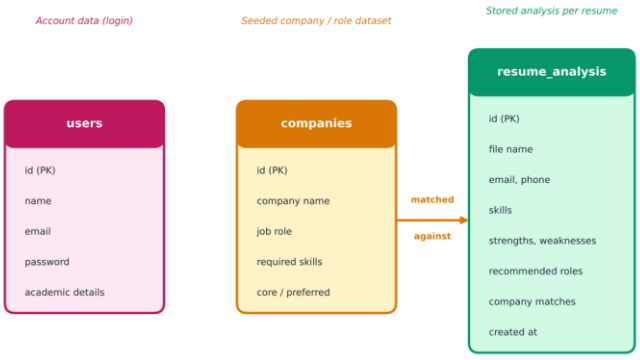



**Fig. 2. Logical data model (key fields of each table)**

The most important design decision is to keep company data in the database instead of in the code. The same records feed the matching algorithm and the recommendation text, so the company list can be extended by adding rows without changing the algorithm. The database path is currently set directly in database.py; moving it to an environment variable is listed as future work.

### *C. Resume Analysis Design*

When a student uploads a resume, the back end follows the steps in Fig. 3. The upload is checked first, so that a file that is not a PDF or is empty is rejected before any analysis starts. The PDF is then written to a temporary file for parsing, and the analysis result is saved to the database. The analysis returns the fields listed in Table III.

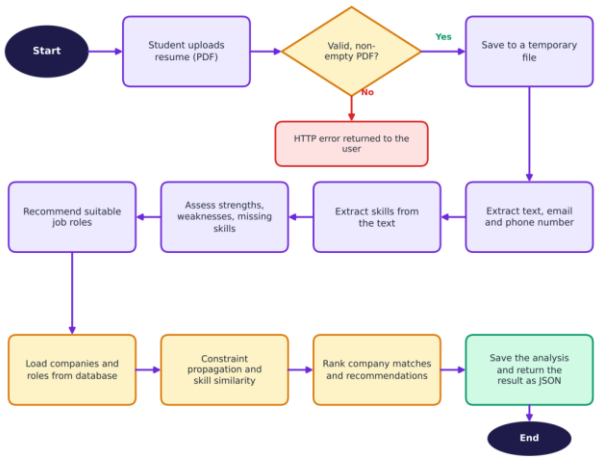



**Fig. 3. Flowchart of the resume analysis pipeline**

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

TABLE III. RESUME ANALYSIS OUTPUT FIELDS

| Field | Description |
|---|---|
| Email and phone | Contact details found in the resume text |
| Skills | Technical and professional skills extracted from the resume |
| Strengths | Areas in which the resume is strong, based on the extracted skills |
| Weaknesses | Areas that are weak or thin in the resume |
| Missing competencies | Skills usually expected for suitable roles that were not found |
| Recommended roles | Job roles that suit the skill profile |
| Company matches | Ranked company roles that fit the profile (Section III-E) |
| Recommendations | Short advice derived from the match results |

## *D. CSP Formulation*

The matching step is formulated as a constraint problem, so that the reasons for keeping or removing a company role can be explained to the student.

### Variables

Each company role stored in the database is a variable. Its value is either *eligible* or *not eligible* for the student who uploaded the resume.

### Domains

Before propagation every variable can take both values. A company role becomes not eligible when a hard constraint fails. The skills in the resume and in the company data are compared after normalisation, so that differences in letter case or punctuation, for example between “Node.js” and “node js”, do not cause false mismatches.

### Constraints

TABLE IV. CONSTRAINTS USED FOR COMPANY MATCHING

| ID | Constraint | Type |
|---|---|---|
| C1 | Every core skill required by a role must appear among the normalised resume skills | Hard |
| C2 | A resume with no extractable skills cannot be matched to any role | Hard |
| C3 | Skill names are compared after normalising case and punctuation | Hard |
| C4 | Preferred skills raise the score of a company role but never remove it | Soft |
| C5 | Company roles with a higher skill similarity are ranked earlier | Soft |

Hard constraints decide which company roles remain. Soft constraints only influence the order of the roles that remain.

## *E. Matching Algorithm*

The matching step works in the following stages, which are also shown in Algorithm 1.

1. **Normalise.** Convert the resume skills and the company skills to a common form (C3).
2. **Load candidates.** Read all company roles from the database.
3. **Propagate constraints.** Remove every company role for which a core skill is missing from the resume (C1, C2).
4. **Score.** For each remaining role compute the similarity between the resume skills and the skills the role lists (C4, C5).
5. **Rank.** Sort the remaining roles by score, highest first.
6. **Recommend.** Return the ranked matches with short recommendations and store them with the analysis.

```
function MATCH_COMPANIES(resume_skills, roles):
  skills <- NORMALISE(resume_skills)
  if skills is empty:
    return empty list              # C2
  candidates <- []
  for role in roles:
    if CORE_SKILLS(role) subset of skills:   # C1
      score <- SIMILARITY(skills, role)     # C4, C5
      add (role, score) to candidates
  sort candidates by score, highest first
  return candidates
```

**Algorithm 1. Constraint propagation and ranking for company matching.**

### Why propagation helps

A company role that fails a core requirement should never appear in the result, however well the other skills match. Removing such roles first keeps the final ranking simple, because every role that survives is one the student can reasonably apply for. It is the same idea as removing impossible values from a domain before searching in a CSP. Fig. 4 illustrates the two steps using five fictional company roles and a resume with the skills Python, SQL, React and Git. Company C and Company D are removed because their core skills (Java, Spring, Docker and Linux) are missing from the resume. The three remaining roles are ranked by the share of their listed skills that the resume covers. The scoring formula in the figure is a simplified illustration; the exact similarity measure is defined in the matching code.

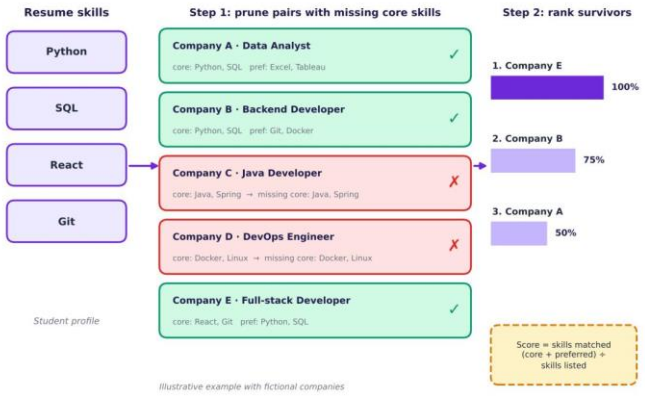



**Fig. 4. Constraint propagation and ranking for company matching (illustrative example)**

In the example of Fig. 4, Company A (Data Analyst) and Company B (Backend Developer) both have core skills Python and SQL, which the resume covers, so both survive; their preferred skills (Excel and Tableau, or Git and Docker) only influence the score. Company E (Full-stack Developer) requires React and Git as core skills and lists Python and SQL as preferred skills, so the resume covers all of its listed skills

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

and it is ranked first. Because the survivors are ranked by the share of listed skills covered, the student sees at once which roles fit best and why the other roles were removed.

**Complexity**

For R company roles with an average of k listed skills, normalisation, the core-skill test and the scoring each take time proportional to R × k, followed by a sort of the m surviving roles in O(m log m) time. The step is a single pass over the data and needs no backtracking, so its running time grows steadily with the size of the company dataset and stays small for a campus-sized list of companies.

### *F. Evaluation and Fallback Design*

Interview answers are evaluated in two ways. A first filter rejects answers that are empty or very short, because they give too little evidence to score. For every other answer, the evaluator uses the OpenAI Chat Completions API when an API key is configured. If no key is present, or the request fails, the local evaluator is used instead. It measures the overlap between the answer and the keywords expected for the question, applies quality heuristics, and returns a score with short feedback.

This design means that the platform never stops at the evaluation step because of a missing key or a network problem. It also keeps a simple and repeatable baseline that can be compared with the language-model score during testing. Because every evaluation returns written feedback together with the score, the student can see what to improve and not only how the answer was rated.

**TABLE V. EVALUATION MODES OF THE MOCK INTERVIEW**

| Aspect | OpenAI mode | Local mode |
|---|---|---|
| Condition | API key configured and request succeeds | No key, or the request fails |
| Method | LLM scores the answer | Keyword overlap with expected terms plus quality heuristics |
| Pre-filter | Empty or very short answers are rejected before scoring | Same pre-filter |
| Output | Score and written feedback | Score and short feedback |

### *G. Key Terminologies*

- **CSP:** variables, domains and constraints whose solution is a consistent assignment.
- **Constraint propagation:** using the constraints to remove values that cannot be part of any solution before further steps.
- **Core skill:** a skill a company role requires; if it is missing the role is removed. **Preferred skill:** a skill that only raises the score.
- **Hard / soft constraint:** a rule that must hold / a preference that influences the order of results.
- **Graceful degradation:** a feature that relies on an external service still returns a useful result when the service is unavailable.

### *H. Design Decisions*

Four design decisions shape the system. First, company data is kept in the database instead of in the code, so the matching algorithm, the recommendation text and the seeding script all use the same records and the list of companies can grow without code changes. Second, hard and soft constraints are kept apart: hard constraints (C1 to C3) decide which company roles remain, while soft constraints (C4 and C5) only influence the order, so a high score on other skills can never hide a missing core skill. Third, the evaluator is dual-mode, so a missing key or a network problem never stops an interview round. Fourth, the front end and back end communicate only through a REST interface that exchanges JSON, which allows them to be run, and in future deployed, separately.

### *I. Request Validation and Error Handling*

Every request to the back end is validated by Pydantic models before any processing starts. Resume uploads are checked first: a file that is not a PDF or is empty is rejected with an HTTP error that is returned to the user, and no analysis is started. A resume with no extractable skills cannot be matched to any role (C2), so the matching step returns an empty list instead of an arbitrary ranking. In the evaluation step, empty or very short answers receive a low score and a request for more detail, and an error from the language-model service leads to the local evaluator instead of a failed request.

## *IV. DETAILED METHODOLOGY*

### *A. Overview*

The system is divided into eight modules. Fig. 5 shows how the student portal sits on top of a shared REST layer and the core services.

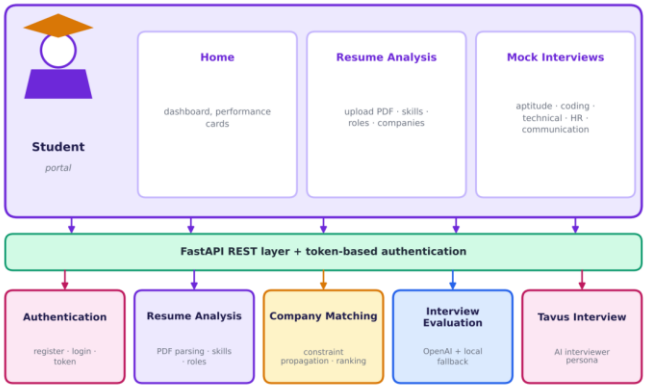



**Fig. 5. Module overview of the system**

### *B. Module 1: Authentication*

This module lets a student create an account and sign in. Registration collects personal academic information together with the email and password. Login checks the email and password and returns an access token that the front end sends with later requests. The input is the registration details or the email and password; processing is validation with Pydantic models, lookup of the user in the database and creation of the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

access token; the output is a success message on registration or an access token on login. In the current prototype, passwords are stored without hashing. This is acceptable for a demonstration but not for production use, and password hashing is the first item of future work. The interfaces of the platform are listed in Table VI.

**TABLE VI. REST INTERFACES OF THE BACK END**

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /api/auth/test | Health check of the authentication API |
| POST | /api/auth/register | Register a new user |
| POST | /api/auth/login | Log in and receive an access token |
| GET | / | Status check of the back end |
| POST | /resume/analyze | Upload and analyse a PDF resume |
| POST | /api/mock-interview/evaluate | Score a free-text answer |
| POST | /api/mock-interview/evaluate-code | Evaluate a coding answer |
| POST | /tavus/conversation | Start the AI interview |

### *C. Module 2: Resume Upload and Parsing*

This module accepts a resume file in PDF format sent to POST /resume/analyze. The file type and content are checked, the PDF is written to a temporary file, and the text, email address and phone number are extracted and passed to the analysis module. Uploads that are not PDF files, or that are empty, are rejected with an error message.

### *D. Module 3: Skill and Profile Analysis*

The resume agent reads the extracted text and builds a profile of the student. It produces the list of skills found in the resume, the strengths of the profile based on the skills and experience found, the weaknesses and missing competencies that show what to learn or add before applying, and the recommended roles that suit the profile. This profile is returned to the student and is also the input of the company-matching module.

### *E. Module 4: Company Matching*

This module applies Algorithm 1 to the skills found in the resume. Its input is the normalised resume skills and the company roles stored in the database. Constraint propagation removes roles whose core skills are missing, and the remaining roles are scored by skill similarity and ranked. The output is a ranked list of company matches with recommendation insights, saved together with the analysis.

### *F. Module 5: Mock Interview Evaluation*

This module scores a free-text answer. It supports five round types: aptitude, coding, technical, behavioural and communication. The request POST /api/mock-interview/evaluate carries the question, the answer and the round type. Empty or very short answers are filtered out; other answers are scored by the OpenAI API when a key is available, or by the local keyword-and-quality evaluator otherwise. The output is a score and written feedback. Fig. 6 shows how the answer is processed.

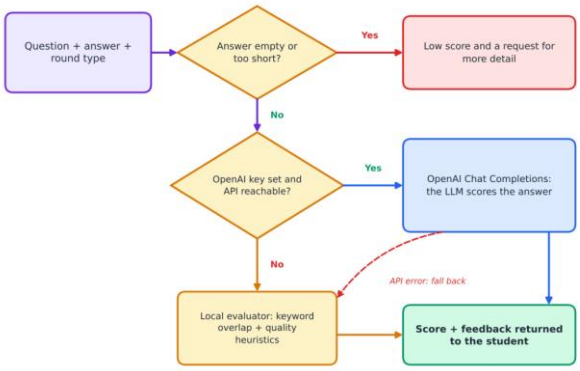



**Fig. 6. Interview answer evaluation with local fallback**

### *G. Module 6: Coding Evaluation*

For the coding round the student submits a solution to a coding problem. The request POST /api/mock-interview/evaluate-code evaluates the coding answer or its test results and returns a score with feedback. As for text answers, the evaluation uses the OpenAI API when it is available and a local fallback when it is not.

### *H. Module 7: Tavus AI Interview*

This module starts a conversational interview in which an AI interviewer talks to the student. The student enters a candidate name and an interview question. The request POST /tavus/conversation sends these values to the back end, which builds a custom interviewer persona and calls the Tavus API with the httpx client. The Tavus key is read from the TAVUS_API_KEY setting in the back-end environment file. Fig. 7 shows the workflow.

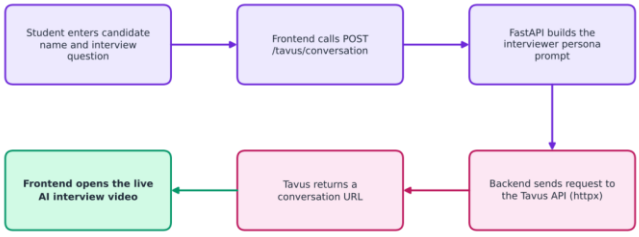



**Fig. 7. Tavus AI interview workflow**

### *I. Module 8: Dashboard and Student Experience*

The front end gives the student one place to use all other modules. It is built with Next.js and has a responsive layout with three sections. **Home** gives an overview with performance cards and progress states. **Resume Analysis** handles upload of the PDF resume and display of the skills, strengths, weaknesses, roles and company matches. **Mock Interviews** lets the student select an interview round, enter answers and see the score and feedback, together with the Tavus interview. Some parts of the interface are still prototype components with placeholder content.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

### J. End-to-End Student Workflow

A typical session combines the modules in the following order.

1. The student registers with personal academic information, an email and a password, and logs in to receive an access token (Module 1).
2. The student uploads a PDF resume on the Resume Analysis page. The file is validated, parsed, and its text, email and phone number are extracted (Module 2).
3. The resume agent returns skills, strengths, weaknesses, missing competencies and recommended roles (Module 3).
4. The company-matching module removes roles with missing core skills and returns the ranked company matches with recommendations, which are stored with the analysis (Module 4).
5. The student opens Mock Interviews, chooses a company and works through its rounds. Text and coding answers are scored with feedback (Modules 5 and 6).
6. The student starts a Tavus session to practise with the conversational AI interviewer (Module 7). All steps are available from the dashboard (Module 8).

### K. Summary of Modules

Table X summarises the input, processing and output of each module. The modules share one REST layer and one database, so the output of resume parsing feeds profile analysis, and the profile feeds company matching.

**TABLE X. INPUT, PROCESSING AND OUTPUT OF THE MODULES**

| Module | Input | Processing | Output |
|---|---|---|---|
| 1 Authentication | Registration details or email and password | Pydantic validation, user lookup, token creation | Success message or access token |
| 2 Upload and parsing | PDF resume | File type and content check, temporary file, text extraction | Text, email and phone number |
| 3 Profile analysis | Extracted resume text | Resume agent reads the text | Skills, strengths, weaknesses, roles |
| 4 Company matching | Normalised skills, company roles | Constraint propagation, similarity scoring, ranking | Ranked matches with insights |
| 5 Answer evaluation | Question, answer, round type | Short-answer filter, OpenAI or local scoring | Score and feedback |
| 6 Coding evaluation | Coding answer or test results | OpenAI or local fallback | Score and feedback |
| 7 Tavus interview | Candidate name, question | Persona prompt, Tavus API call via httpx | Conversation URL |
| 8 Dashboard | Student actions | Calls to the other modules | Home, analysis and interview views |

## V. RESULTS AND DISCUSSION

### A. Experimental Setup

The system was implemented and tested on a development machine. The back end runs with Uvicorn on port 8000 and the Next.js front end runs on its development server on port 3000. Table VII lists the software environment, Table VIII the dataset used for the results, and Table IX the configuration.

**TABLE VII. SOFTWARE ENVIRONMENT**

| Component | Version / tool |
|---|---|
| Python | 3.10 or later |
| Back end framework | FastAPI with Uvicorn |
| Database | SQLite with SQLAlchemy |
| Node.js | 20.9 or later (required by Next.js 16) |
| Front end | Next.js 16, React 19, TypeScript, Tailwind CSS, Framer Motion |
| Language model | OpenAI Chat Completions API (model set in OPENAI_MODEL) |
| Video interview | Tavus API (key set in TAVUS_API_KEY) |

**TABLE VIII. DATASET USED FOR THE EXPERIMENT**

| Item | Value |
|---|---|
| Companies | 10 companies seeded by seed_companies.py |
| Company roles | 4 roles with their required skills |
| Sample resumes | 1 PDF resume |
| Interview rounds | 5 (aptitude, coding, technical, behavioural, communication) |
| Evaluation modes | OpenAI model and local fallback |
| Demo users | 3 registered users |

**TABLE IX. CONFIGURATION AND RUN DETAILS**

| Parameter | Value |
|---|---|
| Back-end port | 8000 |
| Front-end port | 3000 |
| Resume format | PDF |
| Evaluator with an API key | OpenAI Chat Completions |
| Evaluator without an API key | Local keyword-and-quality evaluator |

The experiment follows four steps: seed the database, start the back end and the front end, register and log in, and then upload a resume and attempt the interview rounds while inspecting the outputs.

### B. Student Dashboard

Fig. 8 shows the dashboard. It has sections for Home, Resume Analysis and Mock Interviews, and uses performance cards and progress states to show the student's current position. The Mock Interviews page lists companies such as Google, Amazon, Microsoft and TCS, each with its own sequence of practice rounds.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

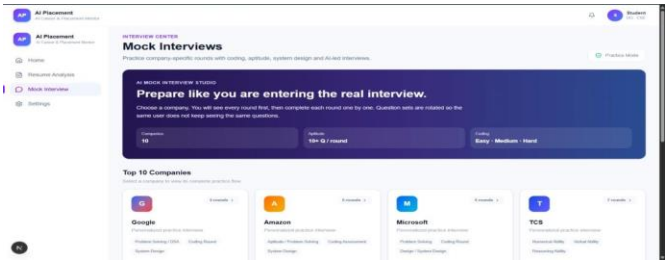



**Fig. 8. Student dashboard (Home)**

## *C. Aptitude Assessment Round*

Fig. 9 illustrates the aptitude assessment phase of the mock interview module. It features timed multiple-choice questions designed to test logical and quantitative reasoning, accompanied by a dynamic progress tracker showing completed and upcoming questions.

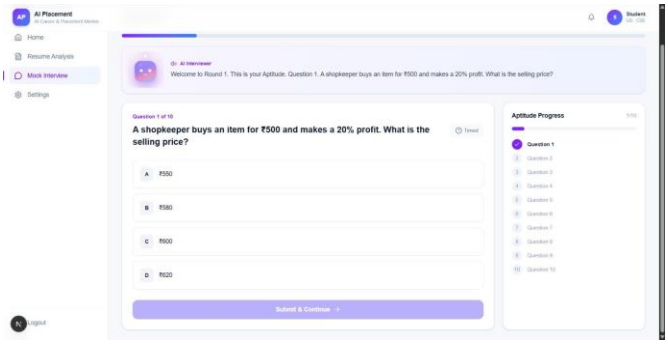



**Fig. 9. Aptitude assessment interface**

## *D. Coding Round*

Fig. 10 displays the interactive coding assessment module within the mock interview platform. It integrates a live code editor with syntax support, problem statement definitions, sample test cases and automated execution testing to evaluate the candidate's programming proficiency.




**Fig. 10. Coding challenge and editor interface**

## *E. Mock Interview Evaluation*

Fig. 11 shows the result screen of a completed practice interview. The student chooses a round type, writes an answer and receives a score with written feedback; the same screen is used when the local evaluator replaces the OpenAI API. In the example, the Infosys practice of 7 rounds was completed with an overall score of 41%.

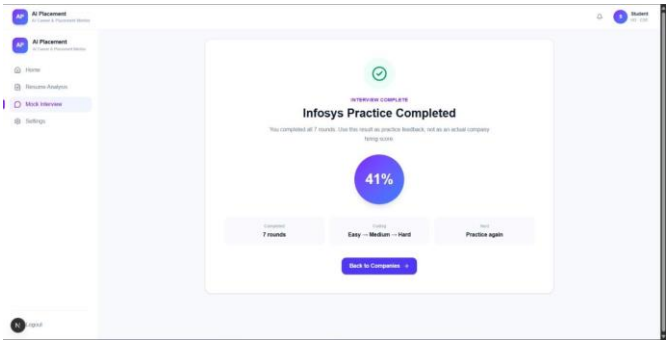



**Fig. 11. Mock interview completion screen with overall score**

## *F. Tavus AI Interview*

Fig. 12 shows a conversational interview with the Tavus AI interviewer. The student enters a name and an interview question, and the interviewer starts the conversation with the persona configured in the back end.

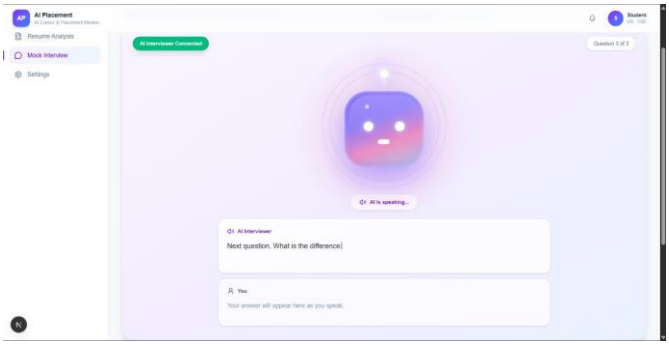



**Fig. 12. Tavus AI interview session**

## *G. Configuration and Security Notes*

The back end reads its external settings from an environment file: the OpenAI key and model name (OPENAI_MODEL) for the language-model evaluator and TAVUS_API_KEY for the Tavus client. Because the evaluator falls back to local scoring when no key is present, the platform can be started and demonstrated without either external service. The front end sends the access token returned at login with later requests, and the back end applies token authentication and CORS settings to its routes. As noted in Section IV-B, the prototype stores passwords without hashing, so it is suitable for demonstration only until the hardening steps in Section VI are carried out.

## *H. Objectives and Observed Outcomes*

Table XI relates each objective of Section I to the outcome observed during the experiment.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

**TABLE XI. OBJECTIVES AND OBSERVED OUTCOMES**

| Objective | Observed outcome |
|---|---|
| Register, sign in, upload PDF resume | Registration and login return an access token; non-PDF and empty uploads are rejected |
| Skills, strengths, weaknesses, roles | Resume agent returns the fields of Table III |
| Company matching with propagation | Roles with missing core skills are removed; the rest are ranked by similarity |
| Five interview round types | Aptitude, coding, technical, behavioural and communication rounds available (Figs. 8–11) |
| OpenAI with local fallback | Same score-and-feedback screen in both modes |
| Conversational AI interviewer | Tavus session starts with the configured persona (Fig. 12) |
| Responsive dashboard | Home, Resume Analysis and Mock Interviews sections (Fig. 8) |

### *I. Discussion*

The platform covers the whole workflow from account creation and PDF resume upload to resume analysis, ranked company matches, five interview round types and a conversational AI interviewer. Constraint propagation keeps the company list explainable: a role is removed only when a stated core skill is missing, and the remaining roles are ordered by skill similarity. The dual-mode evaluator means that the interview rounds remain usable when the OpenAI service cannot be reached.

The two evaluation paths give the student the same kind of output, a score and written feedback, so the interface does not change when the local evaluator replaces the OpenAI API. This matches the graceful-degradation principle described in Section II-D. The language-model path offers natural feedback on free text, while the local path gives a simple and repeatable baseline, as summarised in Table I. The stored company records can also be extended by adding rows, because the same records feed the matching algorithm and the recommendation text.

The results have limits. The current version is a prototype tested on one seeded dataset with a single sample resume, so this paper reports functional behaviour and does not claim quantitative matching accuracy or response-time figures. Passwords are stored without hashing, the database path is fixed in the code, and some interface components still contain placeholder content. These points are addressed in the future work below.

## *VI. CONCLUSION AND FUTURE EXTENSIONS*

This work built a complete placement preparation system around a practical workflow. A student can create an account, upload a PDF resume, receive an analysis of skills, strengths, weaknesses and recommended roles, see ranked company matches, and practise interview rounds with automated feedback and a conversational AI interviewer. The company-matching step uses constraint propagation, a classical AI technique, to remove company roles whose core skills are missing, and then ranks the remaining roles by skill similarity. The interview evaluator combines a language model with a local fallback, so the platform keeps working when the external service is not available. A small, well-understood classical method, combined with a language model and careful software design, can give students a useful and explainable assistant. Future work could extend the system in the following directions:

- Hash passwords with bcrypt or Argon2 and harden the token-based authentication before any production deployment.
- Add role-based access, for example for students, mentors and placement officers.
- Replace the fixed SQLite path with an environment-based database URL.
- Add a more robust resume parser or an OCR step for scanned resumes.
- Build a more advanced recommendation engine on a larger, real job dataset.
- Add analytics and progress tracking for each student across attempts.
- Use structured rubric scoring to improve the quality of interview feedback.
- Deploy the front end and the back end separately.
- Benchmark the platform on larger datasets and record analysis time and matching quality.

## *VII. REFERENCES*


[1] M. Bayer, "SQLAlchemy," in *The Architecture of Open Source Applications, Volume II*, A. Brown and G. Wilson, Eds. Lulu.com, 2012.

[2] T. B. Brown, B. Mann, N. Ryder et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[3] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[4] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[5] K. S. Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[6] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[7] A. K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[8] OpenAI, "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023.

[9] F. Ricci, L. Rokach and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.

[10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[11] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.

[12] E. Tsang, *Foundations of Constraint Satisfaction*. London, U.K.: Academic Press, 1993.

[13] L. Zheng, W.-L. Chiang, Y. Sheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," *Advances in Neural Information Processing Systems*, vol. 36 (Datasets and Benchmarks Track), 2023.


XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE